



Practical Coding Snippets Asked in Interviews



Python – Basic Functional Programs

Q: Write a Python function to check if a string is a palindrome.

```
def is_palindrome(s):  
    return s == s[::-1]  
  
print(is_palindrome("radar")) # True
```

Q: Write a Python program to return the factorial of a number.

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5)) # 120
```

Q: Merge two sorted lists into one sorted list.

```
def merge_sorted_lists(a, b):  
    return sorted(a + b)  
  
print(merge_sorted_lists([1, 3, 5], [2, 4, 6]))
```

JavaScript – Functional Examples

Q: Reverse a string in JavaScript.

```
function reverseString(str) {  
  return str.split('').reverse().join('');  
}  
console.log(reverseString("hello"));
```

Q: Remove duplicates from an array.

```
const removeDuplicates = arr => [...new Set(arr)];  
console.log(removeDuplicates([1, 2, 2, 3, 4, 4]));
```

Q: Manually implement `map()` function.

```
function customMap(arr, callback) {  
  const result = [];  
  for (let i = 0; i < arr.length; i++) {  
    result.push(callback(arr[i], i));  
  }  
  return result;  
}
```

React.js – API Call and List Rendering

Q: Create a component that fetches and displays a list of users.

```
import React, { useEffect, useState } from 'react';

const UserList = () => {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []);

  return (
    <div>
      <h2>User List</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
};

export default UserList;
```

Node.js – Basic API Setup with Express

Q: Create a simple REST API with a GET endpoint.

```
const express = require('express');
const app = express();
const port = 3000;
```

```
app.get('/api/greet', (req, res) => {
  res.json({ message: 'Hello from API!' });
});

app.listen(port, () => {
  console.log(`Server running on http://localhost:${port}`);
});
```

⚡ Next.js – Basic Page with API Call

Q: Display a list of posts fetched from an external API.

```
// pages/index.js
import { useEffect, useState } from 'react';

export default function Home() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch('/api/posts')
      .then(res => res.json())
      .then(data => setPosts(data));
  }, []);

  return (
    <div>
      <h1>Blog Posts</h1>
      <ul>
        {posts.map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

```
    </div>
  );
}
```

Q: Create an API route in Next.js

```
// pages/api/posts.js
export default function handler(req, res) {
  res.status(200).json([
    { id: 1, title: 'First post' },
    { id: 2, title: 'Second post' },
  ]);
}
```

Flask – Simple API Application

Q: Build a Flask app with a single GET API.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/hello', methods=['GET'])
def hello():
    return jsonify({"message": "Hello from Flask!"})

if __name__ == '__main__':
    app.run(debug=True)
```

FastAPI – Basic API Example

Q: Create a FastAPI app with GET and POST endpoints.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.get("/ping")
def read_root():
    return {"message": "pong"}

@app.post("/items/")
def create_item(item: Item):
    return {"item": item}
```

Django – Basic Project and View

Q: Create a Django view to return JSON response.

```
# views.py
from django.http import JsonResponse

def greet(request):
    return JsonResponse({'message': 'Hello from Django!'})
```

Q: Connect this view to a URL path.

```
# urls.py
from django.urls import path
from .views import greet

urlpatterns = [
    path('api/greet/', greet),
]
```

Let me know if you want the same kind of examples for AWS Lambda, Dockerfiles, Terraform, or CI/CD YAMLS!